ASSIGNMENT- 10.5

Name:MD Arshad

HT.No: 2303A51334

Batch: 20

Task Description #1 – Variable Naming Issues

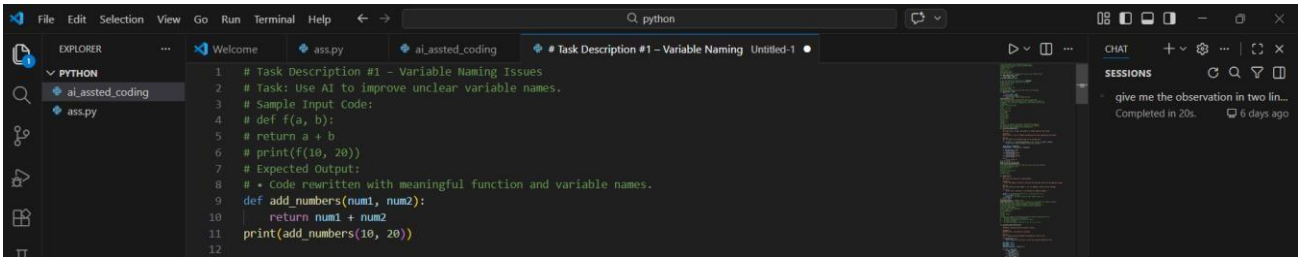
Sample input :

def f(a,b):

return a+b

Print(f(10,20)

Code & probmt



Result:

Observation:

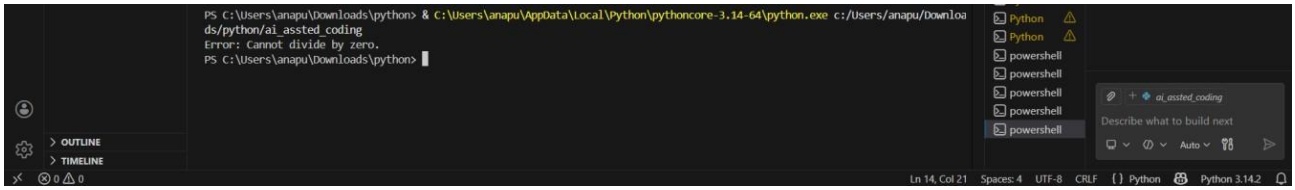
The original code used unclear and non-descriptive names, making it hard to understand the function's purpose. Using meaningful names improves clarity, readability, and overall code quality.

 # Task Description #2 – Missing Error Handling

Sample Input Code:
 # def divide(a, b):
 # return a / b
 # print(divide(10, 0))

Prompt &Code:

Result:



Observation:

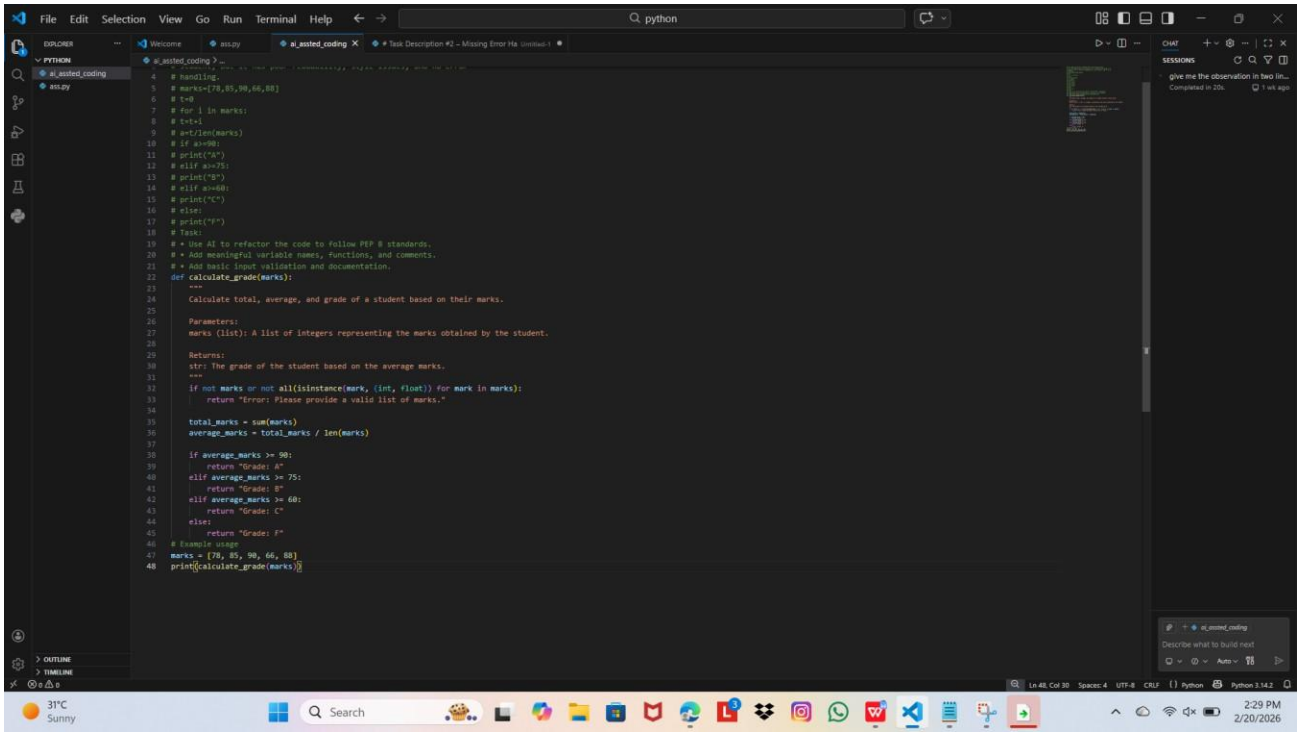
The original code did not handle division by zero, which could cause the program to crash. Adding exception handling makes the program safer and provides a clear, user-friendly error message instead of stopping abruptly.

 # Task Description #3: Student Marks Processing System

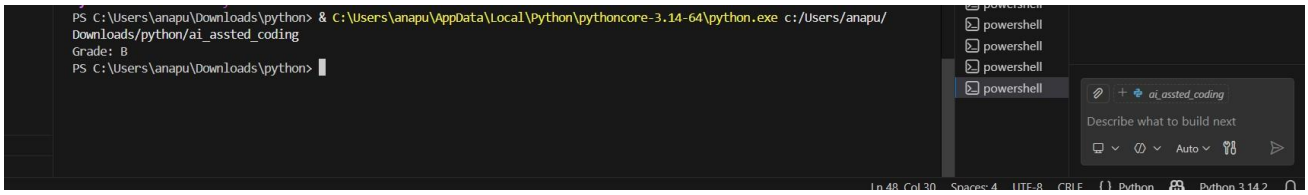
Sample input code:

```
marks=[78,85,90,66,88]
t=0
for i in marks:
t=t+i
a=t/len(marks)
if a>=90:
print("A")
elif a>=75:
print("B")
elif a>=60:
print("C")
else:
print("F")
```

Prompt & Code:



Result:



Observation:

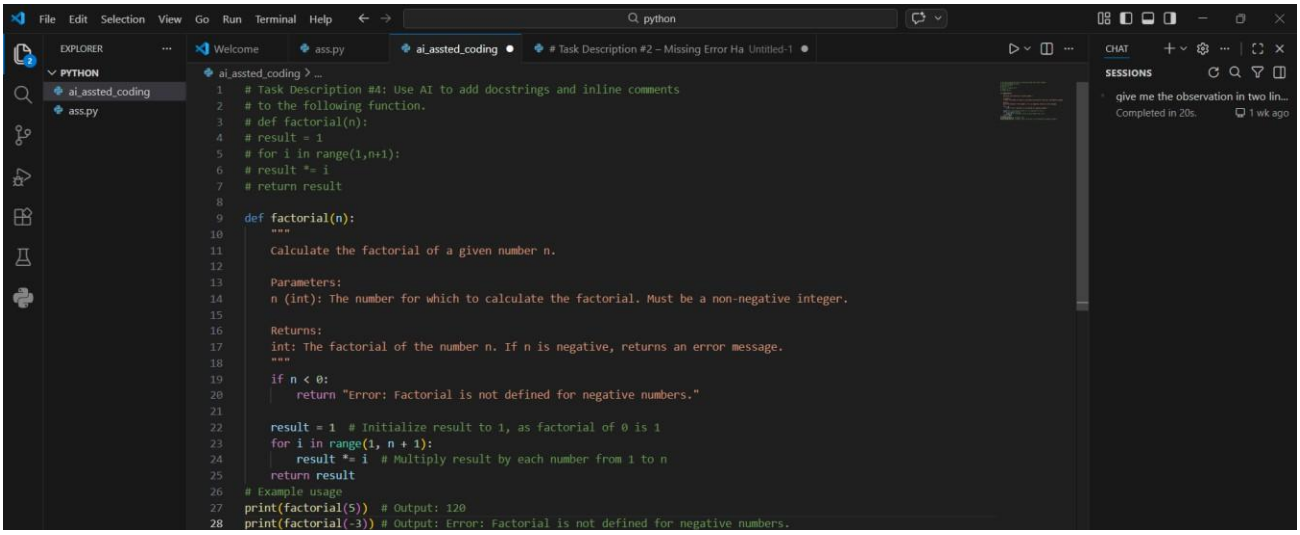
The refactored code improves readability by using meaningful variable names, proper formatting, functions, and clear documentation following PEP 8 standards.
It also adds basic input validation, making the program more reliable and user-friendly by preventing errors from invalid data.

 # Task Description #4: Use AI to add docstrings and inline comments

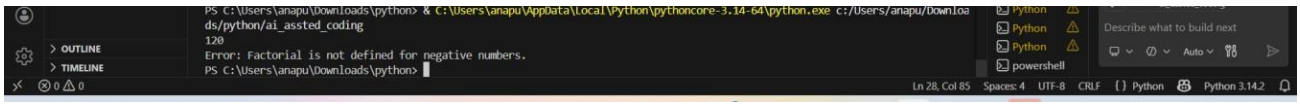
Sample input:

```
def factorial(n):
result = 1
for i in range(1,n+1):
result *= i
return result
```

Prompt &Code:



Result:



Observation

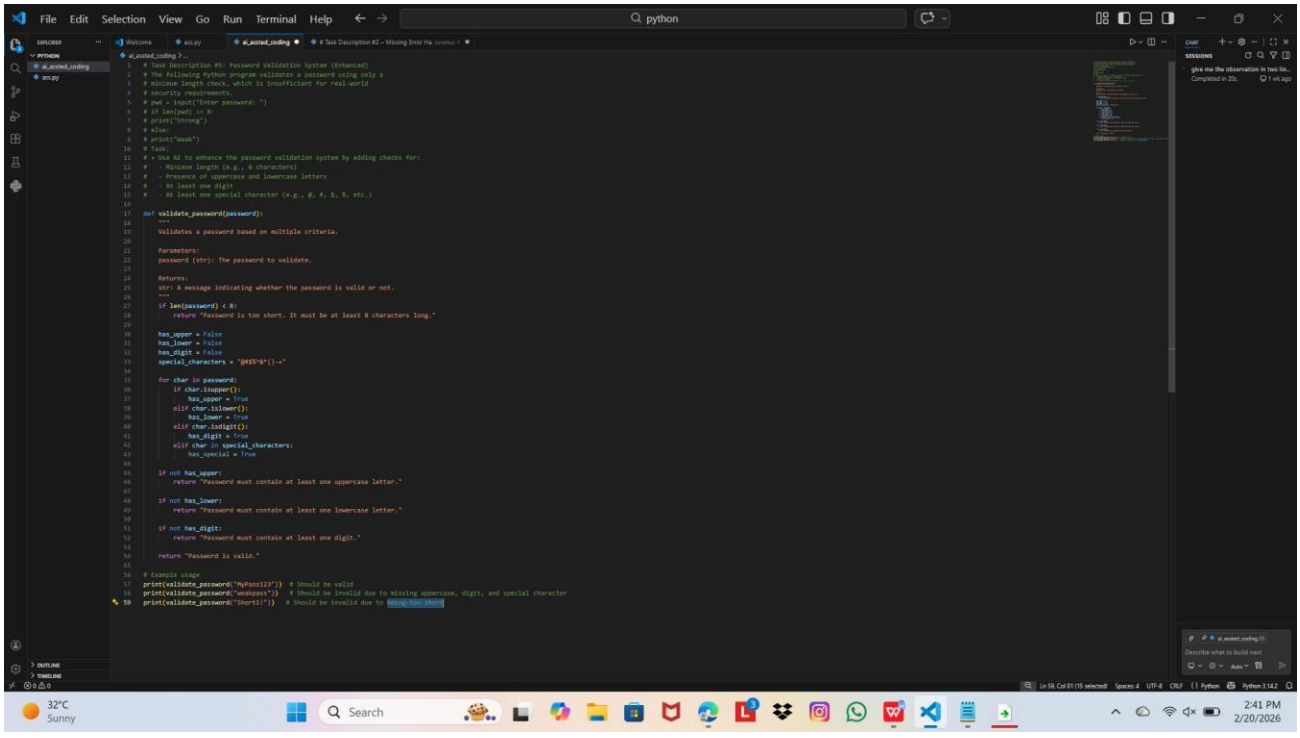
The updated function includes a clear docstring and inline comments, which improve understanding of the code's purpose and logic.
It also adds basic validation for negative numbers, making the function more informative and reliable.

 # Task Description #5: Password Validation System (Enhanced)

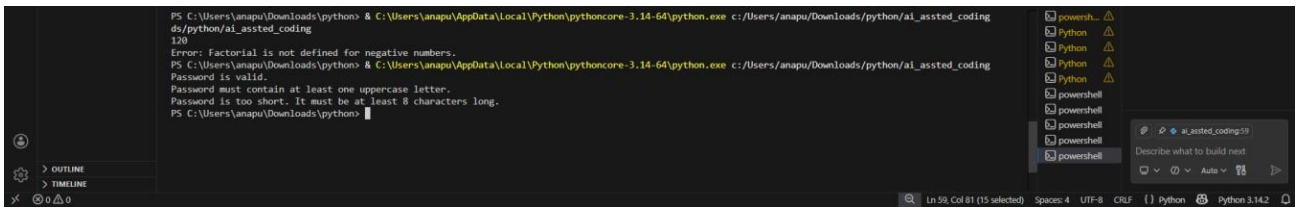
Sample input:

```
pwd = input("Enter password: ")
 if len(pwd) >= 8:  print("Strong")
 else:
 print("Weak")
```

Prombt & Code:



Result:



Observation:

The enhanced password validation improves security by checking for length, uppercase and lowercase letters, digits, and special characters instead of just minimum length.
However, there is a small issue: the variable `has_special` is used without being initialized and the final check for a special character is missing, which could cause an error.